



Introduction

This worksheet demonstrates the process of performing **linear asymmetric quantisation** on a floating-point matrix we looked at in Efficient AI. Similar to last week, you will do some work implementing your own version of the algorithm, to ensure that you understand the details.

1. Import key packages: torch, and any others that you prefer to work with. In general, when writing code, you will put all your import statements at the top. However, for these worksheets we will import as we go along.

```
In [ ]: import torch
```

2. Create a random 4×4 Floating-Point Matrix, which we will quantised next.

```
In [ ]: x = ## TODO ##  
print(x)
```

3. Compute Quantisation Parameters (Scale & Zero-point) of this tensor. \$
Scale = $(x_{\text{max}} - x_{\text{min}}) / (q_{\text{max}} - q_{\text{min}})$ \$, \$ Zero-point = $q_{\text{min}} - \text{round}(\frac{x_{\text{min}}}{s})$ \$

```
In [ ]: # int8 integer range  
q_min, q_max = ## TODO ##  
  
# get data min/max  
x_min = ## TODO ##  
x_max = ## TODO ##  
  
# compute scale and zero-point  
scale = ## TODO ##  
zero_point = ## TODO ##  
  
# clamp zero_point to valid int8 range  
zero_point = ## TODO ##
```

4. Implement the quantisation formula: $q = \text{round}(x / \text{scale} + \text{zero_point})$

```
In [ ]: def quantise(x, scale, zero_point, q_min, q_max):
    # Implement the quantization formula:
    #   q = round(x / scale + zero_point)
    q = ## TODO ##

    # Clamp to valid range: [q_min, q_max]
    q = ## TODO ##

    return q.to(torch.int8)

# quantisation
x_q = quantise(x, scale, zero_point, q_min, q_max)
print("quantised matrix:\n", x_q)
```

5. Implement dequantisation formula: $\text{dequant} = (q - \text{zero_point}) * \text{scale}$

```
In [ ]: def dequantise(q, scale, zero_point):
    # Convert q back to float and apply inverse mapping:
    x_dq = ## TODO ##
    return x_dq

x_dq = dequantise(x_q, scale, zero_point)
print("dequantised matrix:\n", x_dq)
```